TESTE DE SOFTWARE

Detecção e refatoração de Test Smells

Guilherme de Almeida Santos
Matrícula: 800877

Repositório do projeto: github.com/guilhermeas04/test-smelly

18 de outubro de 2026

1. Análise dos Test Smells

A suíte original está em test/userService.smelly.test.js. A execução inicial apresentou quatro testes aprovados e um ignorado. Esse resultado não garante a qualidade das verificações: um teste pode passar mesmo sem detectar uma regressão.

1.1 Lógica condicional no teste

No cenário de desativação (linhas 33-51), um for percorre usuário comum e administrador, enquanto um if/else escolhe quais expectativas executar. O teste mistura duas regras de negócio e acrescenta decisões ao código de verificação. Isso dificulta a leitura e o diagnóstico de falhas; alterações nos dados podem mudar quais asserções são exercitadas. A refatoração criou cenários independentes para permitir a desativação do usuário comum e impedir a do administrador. Cada um verifica o resultado e o estado persistido, sem condições ou laços.

1.2 Tratamento inadequado de exceção

No teste de menor de idade (linhas 66-74), a única asserção está no catch. Se createUser deixar de lançar a exceção, o catch não será executado e o teste passará sem verificar nada. O risco é um falso positivo: uma regressão na regra de idade pode chegar ao produto sem ser detectada. A nova versão usa toThrow com uma função que executa a criação. Assim, a ausência da exceção faz o teste falhar. O antes e depois estão na próxima página.

1.3 Teste ignorado e vazio

O cenário sem usuários (linhas 77-79) utiliza test.skip e contém apenas um TODO. Ele não executa verificações e deixa esse comportamento sem cobertura. Além disso, o título promete uma lista vazia, mas generateUserReport retorna texto. O novo teste está ativo e verifica a mensagem de ausência de usuários. Seu nome descreve o comportamento real do relatório, sem exigir um cabeçalho ou uma formatação completa.

Outros ajustes de clareza

O cenário que criava e buscava um usuário foi dividido em dois testes focados. O relatório passou a verificar a presença dos usuários, seus IDs e o status, evitando fixar pontuação, ordem ou quebras de linha. A exigência de um formato exato só seria adequada se esse formato fosse parte do contrato público.

2. Processo de refatoração

O teste de rejeição de menor de idade foi escolhido por permitir que uma falha de negócio fique oculta. Os trechos abaixo preservam o código executável das duas versões; comentários do original foram omitidos para facilitar a comparação.

Antes - asserção executada somente no catch

```
  test('deve falhar ao criar usuário menor de idade', () => {
    try {
      userService.createUser('Menor', 'menor@email.com', 17);
    } catch (e) {
      expect(e.message).toBe('O usuário deve ser maior de idade.');
}
});
```

Depois - exceção exigida explicitamente

```
  test('deve rejeitar a criação de usuário menor de idade', () => {
    // Arrange
    const nome = 'Menor';
    const  email  =  'menor@teste.com';
    const idade = 17;

    // Act: adia a execução para que a asserção capture a exceção.
    const criarUsuarioMenor = () => userService.createUser(nome, email, idade);

    // Assert
    expect(criarUsuarioMenor).toThrow('O usuário deve ser maior de idade.');
  });
```

Decisões e efeitos

O Arrange explicita os dados relevantes ao cenário, especialmente a idade 17. O Act é representado por uma função: sua chamada fica adiada para que o matcher capture a exceção. O Assert usa toThrow para exigir o erro e conferir a mensagem associada à validação de idade.

A remoção de try/catch elimina a asserção condicional. Agora, se o serviço aceitar o menor de idade, toThrow falhará. O título descreve a regra de negócio e o teste verifica um único comportamento, sem dependência de cenários anteriores.

O setup recria o serviço e limpa o banco em memória antes de cada teste. O helper existente _clearDB é mantido para isolar os dados. O arquivo original e o código de produção não foram modificados.

3. Ferramenta e validação

O comando npx eslint . reportou quatro erros de jest/no-conditional-expect e dois avisos: jest/no-disabled-tests e jest/expect-expect. As linhas 44, 46 e 49 correspondem ao if/else de desativação; a linha 73 corresponde ao catch. Os dois avisos estão na linha 77.

```
PS> npx eslint .

test/userService.smelly.test.js
  44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  77:3  warning  Tests should not be skipped            jest/no-disabled-tests
  77:3  warning  Test has no assertions                 jest/expect-expect

X 6 problems (4 errors, 2 warnings)
```

Figura 1. Reprodução visual da saída real de uma nova execução, com caminho abreviado. Os seis diagnósticos coincidem com o registro inicial.

A ferramenta localiza automaticamente expectativas condicionais, testes desativados e ausência de asserções. Isso confirma parte da análise manual, mas não decide se um teste reúne responsabilidades demais ou exige detalhes indevidos de apresentação. Também não houve título duplicado detectado.

Resultado final

ESLint no arquivo limpo: npx eslint _tests_/userService.clean.test.js --max-warnings=0 terminou com código 0, sem erros ou avisos. O ESLint no projeto completo continua acusando os smells do arquivo original, preservado conforme o enunciado.

Jest: npm test aprovou as duas suítes, com 11 testes aprovados e 1 ignorado, totalizando 12. Os sete testes novos passaram. O teste ignorado pertence à suíte original. Esses resultados não indicam regressão nos cenários exercitados, embora não provem ausência de todos os defeitos possíveis.

4. Conclusão

Testes focados e organizados em Arrange, Act, Assert tornam explícitos a preparação, a ação e o resultado esperado. A separação dos cenários e a exigência correta de exceções reduzem falsos positivos e facilitam localizar regressões. Verificar o comportamento sem fixar detalhes incidentais também reduz manutenção desnecessária.

A análise estática complementa a execução dos testes: encontra padrões arriscados mesmo quando a suíte está verde. Combinadas à revisão manual, essas práticas aumentam a confiança nas mudanças e ajudam a manter o projeto compreensível e sustentável ao longo do tempo.